Homework 11

Rahaf Alabdullah (rahafaa2)

Question 1: Implementing K-means

Random seed: `np.random.default_rng(12345)`. The 10 observations have two independent $N(0,1)$ coordinates, and $K = 2$. Cluster labels are 0 and 1.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(12345)
X = rng.standard_normal((10, 2))
k = 2
n = X.shape[0]
init_labels = np.arange(n) % k
rng.shuffle(init_labels)

m_k = X[init_labels == 0].mean(axis=0), X[init_labels == 1].mean(axis=0)
means = np.vstack(m_k)
x = X[:, None, :] - means[None, :, :]
D = np.sum(x ** 2, axis=2)
updated_labels = np.argmin(D, axis=1)

print(pd.DataFrame( {"initial": init_labels, "updated": updated_labels}))
plt.scatter(X[:, 0], X[:, 1], c=updated_labels)
plt.scatter(means[:, 0], means[:, 1], marker="X", s=100, c='red', label='Means')
plt.legend()
plt.savefig("Q1-part-a.png")

def  my_kmeans(X, K, init_labels, max_iter=100):
    init_labels=init_labels.copy()
    n, p = X.shape
    mean_=np.vstack([X[init_labels == k].mean(axis=0) for k in range(K)])
    for _ in range(max_iter):
        new_labels = np.argmin(np.sum((X[:, None, :] - mean_[None, :, :]) ** 2, axis=2), axis=1)
        if np.array_equal(new_labels, init_labels):
            break
        init_labels = new_labels.copy()
        for k in range(K):
            if np.any(init_labels == k):
                mean_[k] = X[init_labels == k].mean(axis=0)
    cluster_sum=np.sum((X - mean_[new_labels]) ** 2)
    return new_labels, mean_, cluster_sum
final_labels, final_means, final_wss = my_kmeans(X, 2, init_labels)
print("Final labels:\n", final_labels, "\nFinal means:\n", final_means, "\nWSS:\n", final_wss)
plt.figure()
plt.scatter(X[:, 0], X[:, 1], c=final_labels)
plt.scatter(final_means[:, 0], final_means[:, 1], marker="X", s=100, c='red', label='Means')
plt.legend()
plt.savefig("Q1-part-b.png")

X1 = X.copy()
X2 = X.copy()
start1 = np.arange(10) % 2
rng.shuffle(start1)
start2 = np.arange(10) % 2
rng.shuffle(start2)
X1_labels, X1_means, X1_wss = my_kmeans(X1, 2, start1)
X2_labels, X2_means, X2_wss = my_kmeans(X2, 2, start2)
print("Run 1 - Final labels:\n", X1_labels, "\nFinal means:\n", X1_means, "\nWSS:\n", X1_wss)
print("Run 2 - Final labels:\n", X2_labels, "\nFinal means:\n", X2_means, "\nWSS:\n", X2_wss)
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].scatter(X1[:, 0], X1[:, 1], c=X1_labels)
axes[0].scatter(X1_means[:, 0], X1_means[:, 1], marker="X", s=100, c='red', label='Means')
axes[0].set_xlabel("Run 1")
```

```
axes[0].legend()
axes[1].scatter(X2[:, 0], X2[:, 1], c=X2_labels)
axes[1].scatter(X2_means[:, 0], X2_means[:, 1], marker="X", s=100, c='red', label='Means')
axes[1].set_xlabel("Run 2")
axes[1].legend()
plt.savefig("Q1-part-c.png")
```

(a) One iteration from the random initial assignment (initial cluster sizes 5 and 5). The means of the initial clusters, and the assignments after reassigning each observation to its closest mean:

Cluster	$m_{k,1}$	$m_{k,2}$
0	−0.386	0.124
1	0.208	0.059

Observation	0	1	2	3	4	5	6	7	8	9
Initial	0	1	0	0	0	1	1	1	1	0
Updated	0	0	1	0	1	1	0	0	1	1

(b) Running `my_kmeans` from the same initial assignment until the assignments stop changing:

	Final labels	Mean m_0	Mean m_1	WSS
Part (b)	0 0 1 0 1 1 0 0 1 1	(−0.978, 0.499)	(0.800, −0.317)	14.069

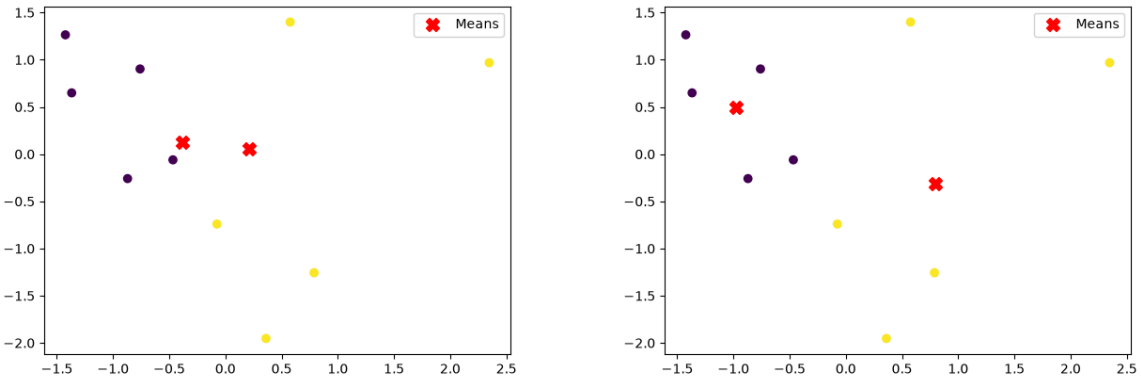


Figure 1: Left: part (a), observations colored by the updated assignments, with the initial cluster means (red). Right: part (b), final clusters and cluster means.

(c) Two runs on the same 10 observations from two different random initial assignments:

Run	Initial labels	Final labels	Mean m_0	Mean m_1	WSS
1	0 1 1 0 1 0 1 0 1 0	0 1 1 0 1 0 0 1 1 0	(−0.126, 1.036)	(−0.053, −0.854)	14.680
2	0 1 0 0 1 0 1 1 0 1	1 1 1 1 1 0 1 1 0 0	(1.237, 0.370)	(−0.658, −0.028)	15.754

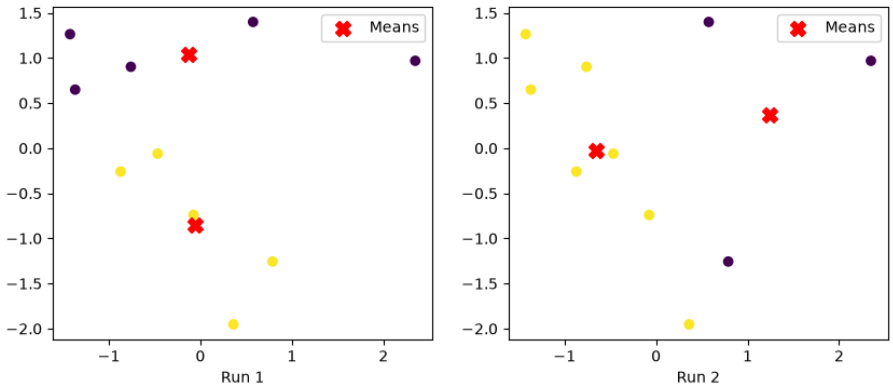


Figure 2: Final clusters and cluster means for the two runs

The K-means objective is non-convex, so different starts CAN end at different groupings with different WSS. With only 10 points and K = 2, run 1 has a lower within-cluster sum of squares than run 2. This is why in practice we use many random starts and keep the run with the smallest within-cluster sum of squares.

Question 2: Image color compression

Random seed: `np.random.default_rng(12345)`. The K-means function is the one from Question 1, with `max_iter=300`. All images and cluster means are on the same $[0, 1]$ RGB scale.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
from pathlib import Path
image_path = Path(__file__).parent / "images" / "cat.jpeg"
name = Path(image_path).stem
out_dir = Path(__file__).parent

rng = np.random.default_rng(12345)
def  my_kmeans(X, K, init_labels, max_iter=300):
    init_labels=init_labels.copy()
    n = X.shape[0]
    mean_=np.vstack([X[init_labels == k].mean(axis=0) for k in range(K)])
    for _ in range(max_iter):
        new_labels = np.argmin(np.sum((X[:, None, :] - mean_[None, :, :]) ** 2, axis=2), axis=1)
        if np.array_equal(new_labels, init_labels):
            break
        init_labels = new_labels.copy()
        for k in range(K):
            if np.any(init_labels == k):
                mean_[k] = X[init_labels == k].mean(axis=0)
    cluster_sum=np.sum((X - mean_[new_labels]) ** 2)
    return new_labels, mean_, cluster_sum

img =Image.open(image_path).convert("RGB")
img.thumbnail((200, 200))
img_arr = np.asarray(img, dtype=float) / 255
H, W = img_arr.shape[:2]

plt.figure()
plt.imshow(img_arr)
plt.axis("off")
plt.title(f"Original image ({H} x {W})")
plt.savefig(out_dir / f"Q2-{name}-original.png")

pixels = img_arr.reshape(-1, 3)
print(f"image is {H} x {W} pixels with 3 channels")

K_list = [2, 3, 5, 10]
for K in K_list:
    n = pixels.shape[0]
    init_labels = np.arange(n) % K
    rng.shuffle(init_labels)
    labels, means, wss = my_kmeans(pixels, K, init_labels)
    compressed_image = means[labels].reshape(H, W, 3)
    print(f"K = {K}, WSS = {wss:.1f}")

    plt.figure()
    plt.imshow(compressed_image)
    plt.axis("off")
    plt.title(f"K = {K}, WSS = {wss:.1f}")
    plt.savefig(out_dir / f"Q2-{name}-compressed-K{K}.png")
```

(a) After resizing, the image is 200 × 133 pixels (height × width) with 3 RGB channels. The pixel data matrix has one row per pixel and the columns red, green, and blue, so its dimensions are **26,600 × 3**.

(b) Cluster means ($[0, 1]$ RGB scale), cluster sizes, and within-cluster sum of squares for each K:

K	Cluster	Pixels	Red	Green	Blue
2	1	16,783	0.368	0.491	0.647
	2	9,817	0.282	0.238	0.175
3	1	3,456	0.742	0.709	0.639
	2	10,012	0.285	0.241	0.180
	3	13,132	0.270	0.434	0.652
5	1	725	0.907	0.702	0.269
	2	4,947	0.412	0.360	0.268
	3	5,472	0.186	0.150	0.120
	4	3,639	0.632	0.683	0.755
	5	11,817	0.249	0.421	0.647

K	Cluster	Pixels	Red	Green	Blue
10	1	709	0.912	0.707	0.269
	2	2,452	0.506	0.604	0.740
	3	3,079	0.298	0.254	0.207
	4	1,229	0.540	0.471	0.382
	5	1,771	0.081	0.060	0.044
	6	5,331	0.291	0.473	0.693
	7	1,235	0.852	0.832	0.810
	8	2,327	0.419	0.373	0.257
	9	2,194	0.210	0.168	0.133
	10	6,273	0.208	0.372	0.605

K	2	3	5	10
WSS	1747.7	944.1	496.8	145.7

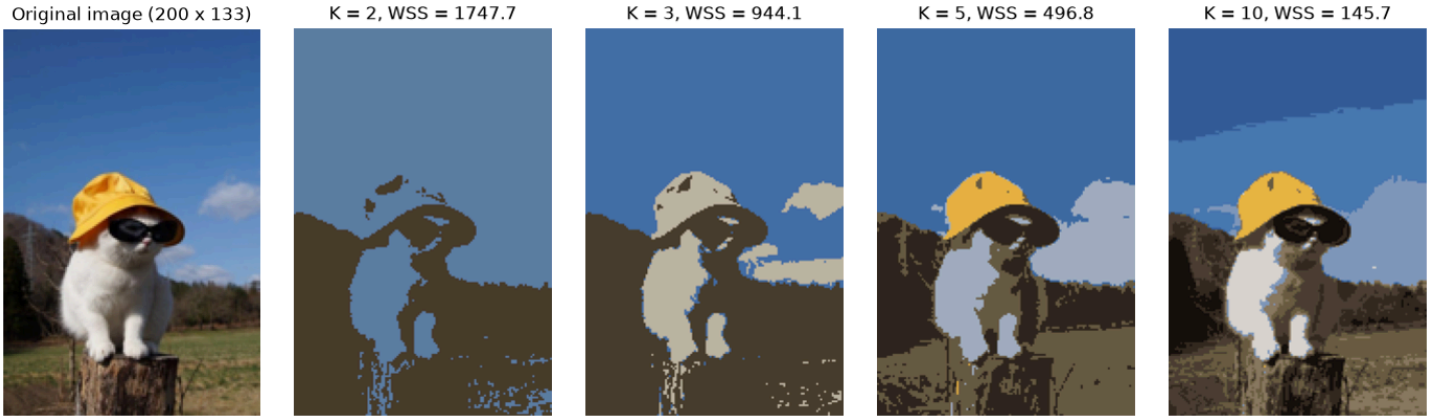


Figure 3: From left to right: the original image (200 × 133), then the compressed images for K = 2, 3, 5, and 10, with each pixel replaced by its cluster mean.

(c) The within-cluster sum of squares (WSS) always goes down as K grows, so smallest WSS alone would always pick the largest K. That isn't a useful rule. K=5 seems to be a good choice for this image, as it provides a good balance between compression and quality. The image retains important details and colors without introducing significant color banding. However, a smaller K means more compression (fewer colors to store); a larger K means better quality.

Question 3: Hierarchical clustering of cars

Each of the 11 numerical variables is standardized to mean 0 and standard deviation 1 (pandas `.std()`, which divides by $n - 1$). The `car` column is used only as the row labels.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster
out_dir = Path(__file__).parent
data_path = out_dir / "data" / "mtcars.csv"

df = pd.read_csv(data_path)
cars = df["car"]
original_df = df.set_index("car")

data_std = (original_df - original_df.mean()) / original_df.std()

linkages = {}
method = ["single", "complete", "average"]
for m in method:
    linkages[m] = linkage(data_std, method=m, metric="euclidean")

fig, axes = plt.subplots(3, 1, figsize=(14, 14))
for i, m in enumerate(method):
    dendrogram(linkages[m], labels=list(original_df.index), ax=axes[i],
               leaf_rotation=90, leaf_font_size=8)
    axes[i].set_title(f"{m.capitalize()} linkage")
    axes[i].set_ylabel("Height (Euclidean distance)")
plt.tight_layout()
plt.savefig(out_dir / "Q3-dendrograms.png")

candidate_linkage = "complete"
Z = linkages[candidate_linkage]
memberships = fcluster(Z, t=4, criterion="maxclust")

clustered_df = original_df.copy()
clustered_df["cluster"] = memberships
for k in range(1, 5):
    cluster_cars = clustered_df[clustered_df["cluster"] == k].index.tolist()
    print(f"Cluster {k} cars: {', '.join(cluster_cars)}")
print("\nCluster sizes:", clustered_df["cluster"].value_counts().sort_index())
print("\nCluster means (original scale):", clustered_df.groupby("cluster").mean().round(2))
```

(a) Dendrograms for single, complete, and average linkage (Euclidean distance on the standardized variables):

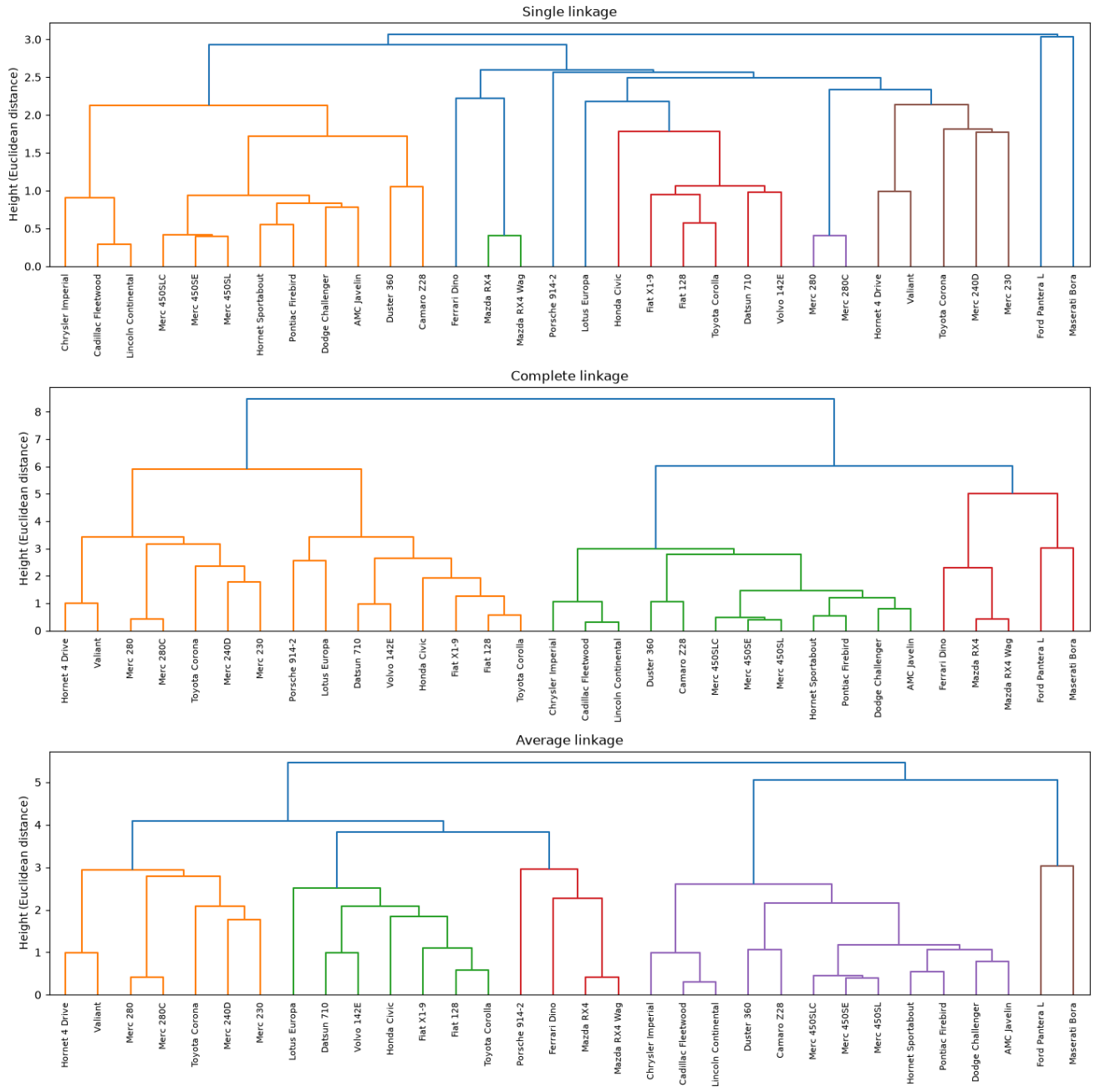


Figure 4: Single, complete, and average linkage dendrograms

For the single linkage dendrogram, the groups are unbalanced and there is a lot of chaining. The complete linkage dendrogram shows more compact and balanced groups with larger jumps in height. The average linkage dendrogram is in between the two, showing some balance but not as compact as complete linkage. Overall, complete linkage seems to provide the most interpretable clusters for this dataset.

(b) Complete linkage, cut into 4 clusters.

Cluster	Size	Cars
1	7	Hornet 4 Drive, Valiant, Merc 240D, Merc 230, Merc 280, Merc 280C, Toyota Corona
2	8	Datsun 710, Fiat 128, Honda Civic, Toyota Corolla, Fiat X1-9, Porsche 914-2, Lotus Europa, Volvo 142E
3	12	Hornet Sportabout, Duster 360, Merc 450SE, Merc 450SL, Merc 450SLC, Cadillac Fleetwood, Lincoln Continental, Chrysler Imperial, Dodge Challenger, AMC Javelin, Camaro Z28, Pontiac Firebird
4	5	Mazda RX4, Mazda RX4 Wag, Ford Pantera L, Ferrari Dino, Maserati Bora

Cluster	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
1	20.74	5.14	175.11	102.14	3.57	3.19	19.97	1.00	0.00	3.57	2.14
2	28.08	4.00	93.61	81.88	4.18	2.04	18.45	0.88	1.00	4.25	1.50
3	15.05	8.00	357.62	194.17	3.12	4.10	17.14	0.00	0.00	3.00	3.08
4	18.50	6.80	223.40	198.80	3.84	3.00	15.62	0.00	1.00	4.60	5.20

Each cluster's means for the big-picture variables (mpg, cyl, disp, hp, wt) show clear differences in size/power vs. economy, while for vs and am, the means represent proportions of cars with manual transmission and straight engines, respectively. The choice of complete linkage and k=4 is justified by the dendrogram's height jump and the distinctiveness of the clusters in terms of their characteristics, making them interpretable and meaningful.

Euclidean distance adds up squared differences across variables, so a variable with a big numeric range dominates. Looking at the raw scales, disp is in the hundreds and hp goes up to ~335, while wt is ~1.5-5.4 and vs/am are just 0/1. Without standardizing, the distances (and so the clusters) would be driven almost entirely by disp and hp, and the other variables would barely count. Standardizing makes each variable count equally.